# Tema 5 (Resumen Total)

Evaluación de Modelos (Clasificación y Regresión), Regularización (Ridge/Lasso), Pipelines + GridSearchCV, y Evaluación de K-Means Clustering

Apuntes de laboratorio (con explicaciones desde cero)

21 de enero de 2026

## Índice


**1. Cómo usar este documento** **4**

**2. Bloque A — Evaluación de Modelos de Clasificación (KNN vs SVM)** **5**
- 2.1. Qué es un problema de clasificación . . . 5
- 2.2. Dataset Breast Cancer (sklearn) . . . 5
- 2.3. Estandarización (StandardScaler): por qué se hace . . . 5
- 2.4. Añadir ruido gaussiano: qué es y por qué se hace . . . 5
  - 2.4.1. Semilla aleatoria (np.random.seed) . . . 6
- 2.5. ¿Qué significa "renderizar" una gráfica? . . . 6
- 2.6. ¿Por qué graficar solo "feature 5"? . . . 6
- 2.7. Split train/test: por qué se hace . . . 6
- 2.8. Matrices de confusión: qué son y cómo leerlas . . . 6
  - 2.8.1. Qué es "positivo" y "negativo" en cáncer . . . 6
- 2.9. Métricas de clasificación (desde cero): accuracy, precision, recall, F1, support . . . 7
  - 2.9.1. Accuracy . . . 7
  - 2.9.2. Precision . . . 7
  - 2.9.3. Recall (Sensibilidad) . . . 7
  - 2.9.4. F1-score . . . 7
  - 2.9.5. Support . . . 7
- 2.10. Train confusion matrix vs Test confusion matrix (y la duda clave) . . . 7
- 2.11. ¿Por qué suelen verse mejores resultados en train que en test? . . . 8
- 2.12. Código base (plantilla) — Clasificación con ruido + KNN y SVM . . . 8

**3. Bloque B — Evaluación de Random Forest (Regresión)** **10**
- 3.1. Qué es regresión (vs clasificación) . . . 10
- 3.2. Random Forest Regressor: idea intuitiva . . . 10
- 3.3. Duda típica: "Si son muchos árboles, por qué veo una recta?" . . . 10
- 3.4. Métricas en regresión . . . 10
  - 3.4.1. MAE . . . 10
  - 3.4.2. MSE . . . 10
  - 3.4.3. RMSE . . . 10
  - 3.4.4. $R^2$ . . . 10
- 3.5. Residuales (errores) y por qué importan . . . 11
- 3.6. Plantilla de código — Random Forest Regression (California Housing) . . . 11

**4. Bloque C — Regularización en Regresión Lineal (Ridge y Lasso) 13**
4.1. Por qué existe la regularización . . . 13
4.2. Ridge (L2) . . . 13
4.3. Lasso (L1) . . . 13
4.4. SNR (Signal-to-Noise Ratio): por qué aparece en la tabla . . . 13
4.5. Interpretación de la tabla (Sparse vs Non-sparse) . . . 14
4.6. Overfitting y MSE: la confusión típica . . . 14
4.7. Sesgo (Bias) y Varianza (Variance): definición clara . . . 14
4.7.1. Sesgo (bias) en ML . . . 14
4.7.2. Varianza . . . 14
4.7.3. ¿Alto sesgo es malo? ¿Bajo sesgo es malo? . . . 14
4.8. Laboratorio típico: outliers + comparación de líneas . . . 14
4.9. Plantilla de código — Regularización (Linear/Ridge/Lasso) . . . 15
4.10. Caso alta dimensión (sparse): por qué Lasso brilla . . . 15

**5. Bloque D — Pipelines y GridSearchCV (metodología imprescindible) 17**
5.1. Qué problema resuelve un Pipeline . . . 17
5.2. Data leakage: definición y ejemplos . . . 17
5.3. Proporciones de clase y stratify: ejemplo exacto . . . 17
5.4. Pipeline + PCA + KNN: explicación (Iris) . . . 18
5.5. GridSearchCV: qué hace y por qué es importante . . . 18
5.6. StratifiedKFold: por qué se usa . . . 18
5.7. Plantilla de código — Pipeline + GridSearchCV (Iris) . . . 18
5.8. Resumen tipo examen: por qué pipeline + gridsearch es “lo correcto” . . . 20

**6. Bloque E — K-Means Clustering y su evaluación 21**
6.1. Qué es clustering (no supervisado) . . . 21
6.2. K-Means: idea intuitiva . . . 21
6.3. Por qué K-Means puede dar resultados distintos (duda típica) . . . 21
6.4. Inercia: qué es y cómo se usa . . . 21
6.5. Elbow method (codo) . . . 21
6.6. Silhouette: explicación desde cero . . . 21
6.7. Davies-Bouldin Index: interpretación . . . 22
6.8. Voronoi (solo para visualizar fronteras) . . . 22
6.9. Plantilla de código — KMeans + métricas (Elbow, Silhouette, DBI) . . . 22
6.10. Resumen de decisión de k (para justificar en examen) . . . 23

**7. Bloque F — Apéndice de “dudas típicas” resueltas 24**
7.1. ¿El ruido crea más o menos datos? . . . 24
7.2. ¿Por qué en algunas gráficas “parece que hay más puntos”? . . . 24
7.3. ¿Qué significa “peor caso” en cáncer? . . . 24
7.4. ¿Qué es “sensibilidad” exactamente? . . . 24
7.5. ¿Por qué precision $\neq$ recall? . . . 24
7.6. ¿Qué es F1 y cuándo tiene sentido? . . . 24
7.7. ¿Qué significa “renderizar” en programación de gráficas? . . . 24
7.8. ¿Qué es data leakage en una frase? . . . 24
7.9. ¿Por qué pipelines son el estándar? . . . 25

**8. Bloque G — “Checklist” final del tema (para repasar rápido) 26**
8.1. Clasificación . . . 26
8.2. Regresión . . . 26
8.3. Regularización . . . 26
8.4. Pipelines + GridSearchCV . . . 26

8.5. Clustering (K-Means) . . . . . 26

# 1. Cómo usar este documento

Este documento está diseñado como un **manual de repaso**:

- Explica **conceptos desde cero** (sin asumir conocimientos previos).
- Incluye **código Python** (en estilo `scikit-learn`) y **por qué se hace cada cosa**.
- Conecta **métricas, gráficas, matrices de confusión, ruido, overfitting, regularización, pipelines** y **selección de hiperparámetros**.

**Nota importante:** cuando hablamos de "tema 5" en estos laboratorios, en realidad estamos juntando varias ideas:

- Evaluación en **clasificación**: accuracy, matriz de confusión, precision, recall, F1, sensitivity.
- Evaluación en **regresión**: MAE, MSE, RMSE, $R^2$, residuales, gráficos actual vs predicted.
- **Regularización**: Ridge (L2), Lasso (L1), efecto de $\lambda/\alpha$.
- **Pipelines + GridSearchCV**: evitar *data leakage*, CV estratificada y elección sistemática de hiperparámetros.
- **Clustering (K-Means)**: inercia (elbow), silhouette, Davies-Bouldin, estabilidad por inicialización.

# 2. Bloque A — Evaluación de Modelos de Clasificación (KNN vs SVM)

## 2.1. Qué es un problema de clasificación

En clasificación, el objetivo es asignar una **clase** a cada ejemplo. Por ejemplo:

- Clase 0: *malignant* (maligno)
- Clase 1: *benign* (benigno)

La salida del modelo NO es un número continuo (como en regresión), sino una etiqueta o clase.

## 2.2. Dataset Breast Cancer (sklearn)

En `sklearn.datasets.load_breast_cancer()` normalmente tenemos:

- $X$: matriz de características (muchas columnas/variables).
- $y$: vector con 0/1 (clase).

## 2.3. Estandarización (StandardScaler): por qué se hace

**Idea:** si una variable mide entre 0 y 1, y otra mide entre 0 y 10 000, la segunda domina las distancias. KNN y SVM son muy sensibles a escalas.

**Qué hace StandardScaler** Transforma cada columna $x$ a:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

donde $\mu$ es la media y $\sigma$ la desviación típica (calculadas sobre el conjunto usado para `fit`).

**Importante (para examen):** no se debe hacer `fit` con todo el dataset antes del split, porque eso produce **data leakage** (lo veremos a fondo en la Sección 5.2). Lo correcto es:

1. split train/test
2. fit scaler en train
3. transform train y test

## 2.4. Añadir ruido gaussiano: qué es y por qué se hace

**Ruido** significa añadir pequeñas perturbaciones aleatorias a los datos. Sirve para simular:

- errores de medida,
- incertidumbre,
- variaciones reales de sensores/datos clínicos.

**Cómo se genera el ruido** Se usa una distribución normal:

$$\epsilon \sim \mathcal{N}(0, 1)$$

Luego se escala con un factor:

$$X_{noisy} = X_{scaled} + \text{noise_factor} \cdot \epsilon$$

**Clave:** el ruido NO cambia el número de filas (pacientes) ni de columnas (features). Cambia los **valores**.

### 2.4.1. Semilla aleatoria (np.random.seed)

`np.random.seed(42)` fija la aleatoriedad para que el experimento sea reproducible: si repites el notebook, obtienes el mismo ruido.

## 2.5. ¿Qué significa "renderizar" una gráfica?

En un notebook, el código prepara la gráfica, pero la gráfica se **muestra** cuando se ejecuta:

- `plt.show()` → **renderiza** (pinta/muestra) la figura en pantalla.

## 2.6. ¿Por qué graficar solo "feature 5"?

Si hay 30 variables, dibujar todas a la vez puede ser caótico. Elegir una sola variable (por ejemplo la 6ª, índice 5) sirve para:

- entender visualmente qué hace el ruido,
- comparar distribución original vs ruidosa,
- ver una "muestra" del efecto sin saturar la vista.

## 2.7. Split train/test: por qué se hace

Separar en entrenamiento y prueba es la base de evaluar generalización:

- **Train**: el modelo aprende (ajusta parámetros).
- **Test**: datos nuevos, para medir rendimiento real.

## 2.8. Matrices de confusión: qué son y cómo leerlas

La matriz de confusión cuenta:

- aciertos,
- errores,
- y qué tipo de error comete el modelo.

Para clasificación binaria, tenemos 4 casos:

| | Pred Positivo | Pred Negativo |
|---|---|---|
| Real Positivo | TP | FN |
| Real Negativo | FP | TN |

### 2.8.1. Qué es "positivo" y "negativo" en cáncer

Por convención:

- Positivo ⇒ **maligno** (hay cáncer).
- Negativo ⇒ **benigno** (no hay cáncer).

Entonces:

- FN (False Negative) = real maligno pero predice benigno ⇒ **peligroso**.

## 2.9. Métricas de clasificación (desde cero): accuracy, precision, recall, F1, support

### 2.9.1. Accuracy

$$\text{Accuracy} = \frac{\#\text{ aciertos}}{\#\text{ total}} = \frac{TP + TN}{TP + TN + FP + FN}$$

**Ojo:** en datasets desbalanceados puede engañar.

### 2.9.2. Precision

**Pregunta:** de todos los que predije como positivos, ¿cuántos lo eran?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Si FP sube, precision baja.

### 2.9.3. Recall (Sensibilidad)

**Pregunta:** de todos los positivos reales, ¿cuántos detecté?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Si FN sube, recall baja.

**Sensibilidad** En medicina, **sensibilidad** = **recall** de la clase positiva (maligno). Cuando en las conclusiones se dice "SVM tiene mayor sensibilidad", significa:

- SVM tiene menos FN, detecta más casos malignos reales.

### 2.9.4. F1-score

Combina precision y recall en una sola métrica (balance).

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

**Idea:** si una es muy baja, F1 cae bastante.

### 2.9.5. Support

**Support** = número de ejemplos reales de una clase en el conjunto evaluado. No mide calidad: solo dice *cuántos datos* había de esa clase.

## 2.10. Train confusion matrix vs Test confusion matrix (y la duda clave)

En el notebook aparecen matrices de confusión de:

- **Train**: se calcula con `X_train` y `y_train`.
- **Test**: se calcula con `X_test` y `y_test`.

**Punto crítico:** si el split se hizo a partir de `X_noisy`, entonces:

$$X_{train}, X_{test} \text{ vienen de } X_{noisy}$$

Es decir, **las dos matrices (train y test) usan datos ruidosos**, pero en subconjuntos distintos.

## 2.11. ¿Por qué suelen verse mejores resultados en train que en test?

Porque el modelo se ha ajustado a los datos de entrenamiento:

- en train el modelo "ya vio" esos patrones,
- en test debe generalizar.

## 2.12. Código base (plantilla) — Clasificación con ruido + KNN y SVM

Listing 1: Plantilla completa: escalado + ruido + split + KNN/SVM + métricas

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 1) Cargar datos
data = load_breast_cancer()
X, y = data.data, data.target
labels = data.target_names
feature_names = data.feature_names

# 2) Escalado (nota: lo correcto es fit SOLO con train; aqui lo dejamos didactico)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3) Ruido gaussiano
np.random.seed(42)
noise_factor = 0.5
X_noisy = X_scaled + noise_factor * np.random.normal(0.0, 1.0, size=X.shape)

# 4) Split
X_train, X_test, y_train, y_test = train_test_split(
    X_noisy, y, test_size=0.3, random_state=42, stratify=y
)

# 5) Modelos
knn = KNeighborsClassifier(n_neighbors=5)
svm = SVC(kernel='linear', C=1, random_state=42)

# 6) Entrenar
knn.fit(X_train, y_train)
svm.fit(X_train, y_train)

# 7) Predecir en TEST
y_pred_knn = knn.predict(X_test)
y_pred_svm = svm.predict(X_test)

# 8) Mtricas
print("KNN test accuracy:", accuracy_score(y_test, y_pred_knn))
print("SVM test accuracy:", accuracy_score(y_test, y_pred_svm))

print("KNN report:\n", classification_report(y_test, y_pred_knn, target_names=labels))
```

```python
print("SVM report:\n", classification_report(y_test, y_pred_svm, target_names=labels))

# 9) Confusion matrices (TEST)
cm_knn = confusion_matrix(y_test, y_pred_knn)
cm_svm = confusion_matrix(y_test, y_pred_svm)

fig, axes = plt.subplots(1,2, figsize=(12,5))
sns.heatmap(cm_knn, annot=True, fmt='d', ax=axes[0], xticklabels=labels, yticklabels=
    labels)
axes[0].set_title("KNN Testing Confusion Matrix")
axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("Actual")

sns.heatmap(cm_svm, annot=True, fmt='d', ax=axes[1], xticklabels=labels, yticklabels=
    labels)
axes[1].set_title("SVM Testing Confusion Matrix")
axes[1].set_xlabel("Predicted"); axes[1].set_ylabel("Actual")
plt.tight_layout(); plt.show()

# 10) Predicciones en TRAIN y matrices de confusin (TRAIN)
y_pred_train_knn = knn.predict(X_train)
y_pred_train_svm = svm.predict(X_train)

cm_knn_train = confusion_matrix(y_train, y_pred_train_knn)
cm_svm_train = confusion_matrix(y_train, y_pred_train_svm)

fig, axes = plt.subplots(1,2, figsize=(12,5))
sns.heatmap(cm_knn_train, annot=True, fmt='d', ax=axes[0], xticklabels=labels,
    yticklabels=labels)
axes[0].set_title("KNN Training Confusion Matrix")
axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("Actual")

sns.heatmap(cm_svm_train, annot=True, fmt='d', ax=axes[1], xticklabels=labels,
    yticklabels=labels)
axes[1].set_title("SVM Training Confusion Matrix")
axes[1].set_xlabel("Predicted"); axes[1].set_ylabel("Actual")
plt.tight_layout(); plt.show()
```

# 3. Bloque B — Evaluación de Random Forest (Regresión)

## 3.1. Qué es regresión (vs clasificación)

En regresión, la variable objetivo $y$ es **continua** (número real). Ejemplo: precio de una casa.

## 3.2. Random Forest Regressor: idea intuitiva

Un Random Forest es un conjunto de muchos árboles de decisión.

- Cada árbol produce una predicción numérica.
- El bosque combina todas las predicciones, típicamente con un promedio:

$$\hat{y}_{forest} = \frac{1}{T}\sum_{t=1}^{T}\hat{y}_t$$

## 3.3. Duda típica: "Si son muchos árboles, por qué veo una recta?"

La gráfica **Actual vs Predicted** no es la función del modelo.

- Es un scatter de puntos: $x$ = valor real, $y$ = valor predicho.
- La línea discontinua es **referencia ideal** $y = x$.

No significa que el modelo sea una recta; es una guía visual.

## 3.4. Métricas en regresión

### 3.4.1. MAE

Error medio absoluto:

$$MAE = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

### 3.4.2. MSE

Error cuadrático medio:

$$MSE = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$$

Penaliza errores grandes.

### 3.4.3. RMSE

$$RMSE = \sqrt{MSE}$$

Vuelve a unidades originales de $y$.

### 3.4.4. $R^2$

Mide proporción de varianza explicada (aprox):

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- $R^2 = 1$ perfecto.
- $R^2 \approx 0$ como predecir la media.
- $R^2 < 0$ peor que predecir la media.

### 3.5. Residuales (errores) y por qué importan

Residual:

$$r_i = y_i - \hat{y}_i$$

Analizar residuales sirve para detectar:

- sesgos: el modelo tiende a sobreestimar o subestimar.
- heterocedasticidad: errores mayores en ciertos rangos.

### 3.6. Plantilla de código — Random Forest Regression (California Housing)

Listing 2: Random Forest Regression: entrenamiento + métricas + gráficos

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1) Cargar dataset
data = fetch_california_housing()
X, y = data.data, data.target # y suele estar en cientos de miles de $

# 2) Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3) Modelo
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# 4) Prediccin
y_pred = rf.predict(X_test)

# 5) Mtricas
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)

# 6) Actual vs Predicted (interpretacin: cercana a y=x)
plt.figure(figsize=(7,6))
plt.scatter(y_test, y_pred, alpha=0.4, edgecolor='k')
m = min(y_test.min(), y_pred.min())
M = max(y_test.max(), y_pred.max())
plt.plot([m, M], [m, M], 'k--', lw=2)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Random Forest Regression - Actual vs Predicted")
plt.tight_layout()
plt.show()
```

```python
# 7) Residuales
residuals = y_test - y_pred
plt.figure(figsize=(7,4))
plt.hist(residuals, bins=30, edgecolor='k')
plt.title("Residuals Histogram")
plt.xlabel("Residual (y - y_pred)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 8) Importancia de variables
importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]
feature_names = data.feature_names

plt.figure(figsize=(10,4))
plt.bar(range(X.shape[1]), importances[indices], align="center")
plt.xticks(range(X.shape[1]), [feature_names[i] for i in indices], rotation=45, ha='right'
    )
plt.title("Feature Importances (Random Forest)")
plt.tight_layout()
plt.show()
```

# 4. Bloque C — Regularización en Regresión Lineal (Ridge y Lasso)

## 4.1. Por qué existe la regularización

La regresión lineal clásica minimiza el error en train:

$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

Con muchas variables, colinealidad o ruido, puede:

- ajustarse al ruido (**overfitting**),
- producir coeficientes muy grandes e inestables.

La regularización añade una penalización:

$$\text{Coste} = RSS + \lambda \cdot \text{Penalización}$$

$\lambda$ **(o `alpha` en sklearn)** controla la fuerza de la penalización:

- $\lambda = 0 \Rightarrow$ regresión normal.
- $\lambda$ grande $\Rightarrow$ coeficientes más pequeños (modelo más simple).

## 4.2. Ridge (L2)

$$\text{Coste}_{Ridge} = RSS + \lambda \sum_j \beta_j^2$$

Propiedades:

- encoge coeficientes,
- rara vez deja coeficientes exactamente en 0,
- es robusto cuando muchas variables aportan "un poco".

## 4.3. Lasso (L1)

$$\text{Coste}_{Lasso} = RSS + \lambda \sum_j |\beta_j|$$

Propiedades:

- encoge coeficientes,
- puede poner coeficientes exactamente en 0 $\Rightarrow$ **selección de variables**.

## 4.4. SNR (Signal-to-Noise Ratio): por qué aparece en la tabla

SNR alto:

- relación señal fuerte,
- poco ruido,
- casi cualquier método va bien.

SNR bajo:

- ruido alto,
- fácil sobreajustar,
- regularización suele mejorar test.

## 4.5. Interpretación de la tabla (Sparse vs Non-sparse)

**Sparse coefficients** significa: muchas $\beta_j$ reales son 0 (solo pocas variables importan). **Non-sparse** significa: muchas variables sí importan (pocos ceros).

- En **sparse + low SNR**, Lasso suele destacar: apaga variables inútiles y reduce ruido aprendido.
- En **non-sparse**, Ridge suele ser muy competitivo: no elimina a lo bruto, solo estabiliza.

## 4.6. Overfitting y MSE: la confusión típica

Cuando decimos **"sobreajuste"**, nos referimos a:

- MSE_train baja mucho,
- pero MSE_test **sube** (generaliza peor).

No es contradictorio: el modelo clava train, pero falla en datos nuevos.

## 4.7. Sesgo (Bias) y Varianza (Variance): definición clara

### 4.7.1. Sesgo (bias) en ML

**Sesgo** es el error por simplificación: el modelo es demasiado simple para capturar la relación real.

- Sesgo alto $\Rightarrow$ underfitting (mal en train y test).
- Sesgo bajo $\Rightarrow$ modelo flexible (puede encajar mejor).

### 4.7.2. Varianza

Varianza es sensibilidad al ruido: cuánto cambia el modelo si cambias un poco los datos.

- Varianza alta $\Rightarrow$ overfitting (train muy bien, test peor).
- Varianza baja $\Rightarrow$ modelo estable.

### 4.7.3. ¿Alto sesgo es malo? ¿Bajo sesgo es malo?

No son "malos" por sí mismos:

- Sesgo alto suele ser malo porque underfit.
- Sesgo muy bajo puede ser malo si viene con varianza alta (overfit).

Lo óptimo es el **equilibrio** que minimiza el error en test.

## 4.8. Laboratorio típico: outliers + comparación de líneas

Un experimento clásico:

- generar datos lineales,
- meter outliers (puntos extremos),
- comparar ajuste de Linear vs Ridge vs Lasso.

## 4.9. Plantilla de código — Regularización (Linear/Ridge/Lasso)

Listing 3: Regularización en regresión lineal: ejemplo autocontenido

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error

np.random.seed(42)
n = 200
X = 2*np.random.rand(n, 1)
noise = 1.0
y = 4 + 3*X + noise*np.random.randn(n,1)

# Aadimos outliers
y_out = y.copy()
out_idx = np.random.choice(np.where(X.flatten()>1.5)[0], size=5, replace=False)
y_out[out_idx] += np.random.uniform(50,100,size=5).reshape(-1,1)

# Modelos
lin = LinearRegression()
rid = Ridge(alpha=1.0)
las = Lasso(alpha=0.2)

lin.fit(X, y_out); y_lin = lin.predict(X)
rid.fit(X, y_out); y_rid = rid.predict(X)
las.fit(X, y_out); y_las = las.predict(X)

# MSE respecto a y "limpia"
print("MSE Linear:", mean_squared_error(y, y_lin))
print("MSE Ridge :", mean_squared_error(y, y_rid))
print("MSE Lasso :", mean_squared_error(y, y_las))

# Grfico
plt.figure(figsize=(10,5))
plt.scatter(X, y_out, alpha=0.35, edgecolor='k', label='Data con outliers')
plt.plot(X, 4 + 3*X, 'g', lw=3, label='Relacin ideal (sin ruido)')
plt.plot(X, y_lin, lw=2, label='Linear')
plt.plot(X, y_rid, '--', lw=2, label='Ridge')
plt.plot(X, y_las, ':', lw=3, label='Lasso')
plt.legend()
plt.title("Efecto de regularizacin con outliers")
plt.tight_layout()
plt.show()

# Coeficientes
print("Coef Linear:", lin.coef_.ravel(), "Intercept:", lin.intercept_.ravel())
print("Coef Ridge :", rid.coef_.ravel(), "Intercept:", rid.intercept_.ravel())
print("Coef Lasso :", las.coef_.ravel(), "Intercept:", las.intercept_.ravel())
```

## 4.10. Caso alta dimensión (sparse): por qué Lasso brilla

Si generamos un dataset con:

- 100 variables,
- solo 10 informativas,
- ruido alto,

entonces Lasso tiende a:

- poner muchas $\beta$ en 0,
- seleccionar las features relevantes,
- mejorar interpretabilidad y a veces test.

Listing 4: Dataset sparse + ruido: comparar coeficientes (Linear/Ridge/Lasso)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error

X, y, coef_true = make_regression(
    n_samples=120, n_features=100, n_informative=10,
    noise=10, coef=True, random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

lin = LinearRegression()
rid = Ridge(alpha=1.0)
las = Lasso(alpha=0.1)

lin.fit(X_train, y_train)
rid.fit(X_train, y_train)
las.fit(X_train, y_train)

y_lin = lin.predict(X_test)
y_rid = rid.predict(X_test)
y_las = las.predict(X_test)

print("MSE Linear:", mean_squared_error(y_test, y_lin))
print("MSE Ridge :", mean_squared_error(y_test, y_rid))
print("MSE Lasso :", mean_squared_error(y_test, y_las))

# Comparar coeficientes estimados vs verdaderos
plt.figure(figsize=(10,4))
plt.plot(coef_true, label="Coef verdadero", lw=2)
plt.plot(lin.coef_, label="Linear", alpha=0.8)
plt.plot(rid.coef_, label="Ridge", alpha=0.8)
plt.plot(las.coef_, label="Lasso", alpha=0.9)
plt.title("Coeficientes: true vs estimados")
plt.legend()
plt.tight_layout()
plt.show()

# Cuantos ceros pone Lasso
num_zeros = np.sum(np.isclose(las.coef_, 0))
print("Coeficientes exactamente 0 (Lasso):", num_zeros, "de", las.coef_.shape[0])
```

# 5. Bloque D — Pipelines y GridSearchCV (metodología imprescindible)

## 5.1. Qué problema resuelve un Pipeline

Un pipeline encadena:

$$\text{preprocesado} \rightarrow \text{modelo}$$

en un único objeto. Ventajas:

- reduce errores humanos,
- asegura que train y test reciben el **mismo** preprocesado,
- permite que GridSearchCV ajuste hiperparámetros de todo el flujo,
- evita **data leakage** en validación cruzada.

## 5.2. Data leakage: definición y ejemplos

**Data leakage** (fuga de información) ocurre cuando información del test (o del futuro) se usa durante el entrenamiento.

Ejemplo típico:

- Hacer `scaler.fit(X)` antes de hacer split.

Eso usa estadísticas del test (media, desviación) **durante el entrenamiento**.

### Correcto

- split
- fit scaler en train
- transform train y test

**Pipeline lo hace por ti** Cuando llamas `pipeline.fit(X_train, y_train)`, el pipeline:

- hace `fit` de los pasos con train,
- y cuando haces `predict(X_test)`, solo hace `transform` con parámetros aprendidos en train.

## 5.3. Proporciones de clase y stratify: ejemplo exacto

Supón 100 muestras:

- clase 0: 80 muestras (80 %)
- clase 1: 20 muestras (20 %)

Si haces test_size=0.2 (20 muestras en test), idealmente quieres:

- test: 16 de clase 0 y 4 de clase 1
- train: 64 de clase 0 y 16 de clase 1

`stratify=y` fuerza a mantener esas proporciones.

Listing 5: train_test_split estratificado y comprobación de proporciones

```
import numpy as np
from sklearn.model_selection import train_test_split

# Ejemplo artificial
y = np.array([0]*80 + [1]*20)
X = np.random.randn(100, 3)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Total:", np.bincount(y)) # [80 20]
print("Train:", np.bincount(y_train)) # aprox [64 16]
print("Test :", np.bincount(y_test)) # aprox [16 4]
```

## 5.4. Pipeline + PCA + KNN: explicación (Iris)

En el lab:

- **StandardScaler**: escala.
- **PCA**: reduce dimensionalidad.
- **KNN**: clasifica por vecinos.

## 5.5. GridSearchCV: qué hace y por qué es importante

**GridSearchCV** prueba muchas combinaciones de hiperparámetros usando **validación cruzada (CV)**:

- Divide el train en varios folds.
- Entrena con algunos folds y valida con el fold restante.
- Repite y promedia.
- Elige la combinación con mejor rendimiento promedio.

**Ventaja** No eliges $k$ o #componentes de PCA "porque sí"; lo eliges con evidencia.

## 5.6. StratifiedKFold: por qué se usa

En clasificación multiclase o desbalanceada, quieres que cada fold tenga proporciones similares de clases. Eso hace que la validación sea más estable y justa.

## 5.7. Plantilla de código — Pipeline + GridSearchCV (Iris)

Listing 6: Pipeline + GridSearchCV completo y correcto

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
```

```python
from sklearn.metrics import confusion_matrix

# 1) Datos
data = load_iris()
X, y = data.data, data.target
labels = data.target_names

# 2) Split estratificado
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3) Pipeline (sin hiperparametros fijos, los buscara GridSearch)
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA()),
    ('knn', KNeighborsClassifier())
])

# 4) Grid de hiperparametros (paso__param)
param_grid = {
    'pca__n_components': [2, 3, 4],
    'knn__n_neighbors': [3, 5, 7, 9]
}

# 5) CV estratificada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 6) GridSearchCV
search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
    verbose=2
)

# 7) Entrenar busqueda en TRAIN
search.fit(X_train, y_train)

print("Mejores parametros:", search.best_params_)
print("Mejor score CV:", search.best_score_)

# 8) Evaluacion final en TEST
test_score = search.score(X_test, y_test)
print("Test accuracy:", test_score)

# 9) Matriz de confusion en TEST
y_pred = search.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
           xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix (Best Pipeline)")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```

### 5.8. Resumen tipo examen: por qué pipeline + gridsearch es "lo correcto"

- Evita **data leakage** (el preprocesado se ajusta solo con train en cada fold).
- Hace el proceso reproducible y menos propenso a errores.
- Permite ajustar hiperparámetros de forma sistemática.
- Representa el flujo profesional en ML real.

# 6. Bloque E — K-Means Clustering y su evaluación

## 6.1. Qué es clustering (no supervisado)

En clustering no hay etiquetas $y$ (o no se usan). El objetivo es agrupar puntos similares.

## 6.2. K-Means: idea intuitiva

1. Elegir $k$ centroides iniciales.
2. Asignar cada punto al centroide más cercano.
3. Recalcular centroides como media de puntos asignados.
4. Repetir hasta converger.

## 6.3. Por qué K-Means puede dar resultados distintos (duda típica)

Porque los centroides iniciales pueden ser diferentes. Por eso se repite varias veces o se controla con `random_state`.

## 6.4. Inercia: qué es y cómo se usa

Inercia:

$$\text{Inertia} = \sum_i \|x_i - \mu_{c(i)}\|^2$$

Mide compactación: más bajo suele ser mejor.

## 6.5. Elbow method (codo)

Se grafica inercia vs $k$:

- baja mucho al principio,
- luego mejora poco,

El "codo" es un $k$ razonable.

## 6.6. Silhouette: explicación desde cero

Para cada punto:

- $a(i)$: distancia media a puntos de su propio cluster.
- $b(i)$: mínima distancia media a puntos del cluster vecino más cercano.

Silhouette del punto:

$$s(i) = \frac{b(i) - a(i)}{\text{máx}(a(i), b(i))}$$

Interpretación:

- cercano a 1: bien asignado.
- cercano a 0: frontera.
- negativo: probablemente mal asignado.

**silhouette_score** es el promedio.

## 6.7. Davies-Bouldin Index: interpretación

Davies-Bouldin (DBI) mide similitud entre clusters:

- **más bajo** suele ser mejor (clusters compactos y separados).

## 6.8. Voronoi (solo para visualizar fronteras)

Las regiones de Voronoi muestran "territorio" de cada centroide: cada punto del plano se asignaría al centroide más cercano.

## 6.9. Plantilla de código — KMeans + métricas (Elbow, Silhouette, DBI)

Listing 7: KMeans: generar blobs + evaluar k con inertia/silhouette/DBI

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score, davies_bouldin_score

# 1) Datos 2D para visualizar
X, y_true = make_blobs(
    n_samples=500, centers=4, n_features=2,
    cluster_std=[1.0, 3, 5, 2], random_state=42
)

# 2) Probar distintos k
k_values = range(2, 11)
inertias = []
sil_scores = []
db_scores = []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X, labels))
    db_scores.append(davies_bouldin_score(X, labels))

# 3) Graficas
plt.figure(figsize=(12,3))

plt.subplot(1,3,1)
plt.plot(list(k_values), inertias, marker='o')
plt.title("Elbow (Inertia vs k)")
plt.xlabel("k"); plt.ylabel("Inertia")

plt.subplot(1,3,2)
plt.plot(list(k_values), sil_scores, marker='o')
plt.title("Silhouette vs k")
plt.xlabel("k"); plt.ylabel("Silhouette")

plt.subplot(1,3,3)
plt.plot(list(k_values), db_scores, marker='o')
plt.title("Davies-Bouldin vs k")
plt.xlabel("k"); plt.ylabel("DB Index")

plt.tight_layout()
plt.show()
```

```python
# 4) Entrenar con un k elegido (ejemplo k=4)
k = 4
km = KMeans(n_clusters=k, random_state=42)
labels = km.fit_predict(X)
centers = km.cluster_centers_

plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c=labels, alpha=0.5, edgecolor='k')
plt.scatter(centers[:,0], centers[:,1], s=200, marker='X', edgecolor='k')
plt.title("KMeans clustering (k=4)")
plt.tight_layout()
plt.show()

print("Silhouette(k=4):", silhouette_score(X, labels))
print("DBI(k=4):", davies_bouldin_score(X, labels))
```

## 6.10. Resumen de decisión de k (para justificar en examen)

No existe un único "k correcto" universal. Se justifica con:

- Elbow: un k donde la mejora en inercia se vuelve pequeña.
- Silhouette: elegir k con score alto.
- DBI: elegir k con score bajo.
- Visualización 2D (si aplica) y coherencia del problema.

# 7. Bloque F — Apéndice de "dudas típicas" resueltas

## 7.1. ¿El ruido crea más o menos datos?

No. **El ruido NO cambia el número de filas**. Si $X$ es de tamaño $(n, p)$, entonces $X_{noisy}$ también es $(n, p)$. El ruido solo cambia los valores.

## 7.2. ¿Por qué en algunas gráficas "parece que hay más puntos"?

Porque el ruido aumenta la dispersión: los puntos ocupan más área. Pero son el mismo número.

## 7.3. ¿Qué significa "peor caso" en cáncer?

**Falso Negativo (FN)**: el paciente es maligno pero el test dice benigno. Es grave porque no se trata el cáncer.

## 7.4. ¿Qué es "sensibilidad" exactamente?

Sensibilidad = Recall de la clase positiva:

$$\text{Sensibilidad} = \frac{TP}{TP + FN}$$

Menos FN $\Rightarrow$ más sensibilidad.

## 7.5. ¿Por qué precision $\neq$ recall?

Porque contestan a preguntas distintas:

- Precision: ¿cuántos de mis positivos predichos eran verdaderos? (FP afecta)
- Recall: ¿cuántos positivos reales detecté? (FN afecta)

En medicina suele priorizarse recall/sensibilidad para no perder casos reales.

## 7.6. ¿Qué es F1 y cuándo tiene sentido?

F1 combina precision y recall. Tiene sentido cuando:

- quieres balance (no solo accuracy),
- hay desbalance,
- FP y FN importan ambos.

## 7.7. ¿Qué significa "renderizar" en programación de gráficas?

Es "dibujar en pantalla". En matplotlib suele ser `plt.show()`.

## 7.8. ¿Qué es data leakage en una frase?

Usar información del test durante el entrenamiento (directa o indirectamente), inflando resultados.

## 7.9. ¿Por qué pipelines son el estándar?

Porque automatizan correctamente:

- fit/transform solo con train,
- preprocesado consistente,
- búsqueda de hiperparámetros segura,
- reproducibilidad.

# 8. Bloque G — "Checklist" final del tema (para repasar rápido)

## 8.1. Clasificación

- Split train/test y por qué.
- StandardScaler y por qué.
- Ruido: qué es y por qué se añade.
- Confusion matrix: TP/TN/FP/FN.
- Accuracy vs Precision vs Recall vs F1 vs Support.
- Sensibilidad = Recall de la clase positiva.
- Comparación train vs test para detectar overfitting.

## 8.2. Regresión

- Random Forest = promedio de muchos árboles.
- Métricas: MAE, MSE, RMSE, $R^2$.
- Scatter actual vs predicted (línea y=x como referencia).
- Residuales: histograma y patrón vs valores reales.
- Feature importances: interpretación cuidadosa.

## 8.3. Regularización

- Ridge (L2) encoge coeficientes, no pone ceros exactos.
- Lasso (L1) puede poner coeficientes a 0 (selección).
- $\lambda/\alpha$ controla fuerza.
- Bias-Variance: regularización aumenta sesgo y reduce varianza.
- SNR alto vs bajo; sparse vs non-sparse.

## 8.4. Pipelines + GridSearchCV

- Pipeline evita leakage y errores.
- GridSearchCV prueba hiperparámetros con CV.
- StratifiedKFold mantiene proporciones de clase en folds.
- `step__param` para hiperparámetros dentro del pipeline.

## 8.5. Clustering (K-Means)

- KMeans depende de inicialización.
- Inertia (elbow), silhouette (alto mejor), Davies-Bouldin (bajo mejor).
- Justificación de k con varias métricas + coherencia.

**Fin del documento.**